

Software Testing Assignment

Module 2 — Manual Testing

1) What is Exploratory Testing?

Exploratory Testing is a type of software testing in which the tester is free to select any possible methodology to test the software. It checks the functionality and operations of the software as well as identifies the functional and technical faults in it.

2) What is Traceability Matrix?

A Traceability Matrix is a document that correlates any two baseline documents that require a many-to-many relationship to check the completeness of the relationship. It is used to track the requirements and to check whether the current project requirements are met.

3) What is Boundary Value Testing?

Boundary Value Testing is one of the most widely applied black box testing methodologies. It is defined as a black box test technique that identifies the errors and bugs between the extreme limits or boundaries of a software program or module.

4) What is Equivalence Partitioning Testing?

In equivalence partitioning, equivalence classes are evaluated for given input conditions. Whenever any input is given, the type of input condition is checked, then for these input conditions, an Equivalence class represents a set of valid or invalid states.

The valid partition is bounded by the values 1 and 100. Plus there are 2 invalid partitions.

5) What is Integration Testing?

Integration Testing is a software testing technique that focuses on verifying the interactions and data exchange between different components or modules of a software application. The goal of integration testing is to identify any problems or bugs that arise when different components are combined and interact with each other.

6) What is Alpha Testing?

In-house testing of a product and team testing before launching.

7) What is Beta Testing?

Software or web application testing by outdoor testers before launching the application.

8) What is Component Testing?

A component is a software item that can be tested separately or in isolation. It is the smallest testable part of software. Unit testing is done by the developer side and involves testing of individual software components.

9) What is Functional System Testing?

In functional testing we check whether all functionalities are working properly and fulfil the user's requirements. We also check line by line, each and every step manually.

10) What is Non-Functional Testing?

Non-Functional Testing involves testing the attributes of a component or system that do not relate to functionality — e.g., reliability, efficiency, usability, interoperability, maintainability, and portability. It is performed after functional testing.

Types of Non-Functional Testing:

- 1) Performance Testing
- 2) Load Testing
- 3) Migration Testing
- 4) Stress Testing
- 5) Penetration Testing
- 6) Installation Testing
- 7) Security Testing
- 8) Compatibility Testing
- 9) Volume Testing

11) What is GUI Testing?

GUI stands for Graphical User Interface. GUI testing is the process of testing the system's user interface. It involves checking screens with controls like buttons, menus, checkboxes, icons, toolbars, menu bars, dialog boxes, etc.

12) What is Adhoc Testing?

The main aim of Adhoc testing is to find defects by random checking. It can be achieved with a technique called Error Guessing, where experienced testers think of situations in which the software may not be able to cope.

13) What is Load Testing?

Load testing is a performance testing technique used to check system behavior under load. It determines a system's performance under real-life load conditions and helps determine how the application behaves when multiple users access it simultaneously.

14) What is Stress Testing?

Stress testing is used to test the stability and reliability of the system. The system is stressed beyond its specifications to check how and when it fails. It is performed under

heavy load like large number of records beyond storage capacity, complex database queries, or continuous input to the system. Stress testing is also known as Endurance Testing.

15) What is White Box Testing? List the Types.

White Box Testing is a software testing technique that involves testing the internal structure and workings of a software application. It is also known as Structural Testing or Code-Based Testing and is used to test the software's internal logic, flow, and structure.

Types of White Box Testing:

- Statement Coverage
- Decision Coverage
- Condition Coverage

16) What is Black Box Testing? What are the Different Techniques?

In Black Box Testing, the tester is not concerned with the software's internal knowledge or implementation details, but rather focuses on validating the functionality based on the provided specifications or requirements.

Black Box Testing Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Use-Case Testing

17) What is Big Bang Testing?

Integration testing is a type of testing used to check the functionality of integrated components of a software system. It is performed after unit testing and before validation testing. Individual software components are combined and tested as a group. Integration testing can be either Bottom-Up or Top-Down.

18) What is the Purpose of Exit Criteria?

Exit criteria for STLC phases are actions or tasks that must be finished before the current phase is over and the next one begins. The following requirements must be fulfilled for the test cases development phase to be completed: test cases must be written and examined, and test data needs to be prepared and recognized.

19) When Should Regression Testing be Performed?

When any application has updated its data or any initial build is done, we need to check whether other options and functions are working properly or not. This is called Regression Testing.

20) What are the 7 Key Principles of Testing? Explain in Detail.

- 1) **Testing Shows Presence of Defects:** Testing can reveal bugs, but cannot prove that the software is 100% defect-free.
- 2) **Exhaustive Testing is Impossible:** We cannot test everything.
- 3) **Early Testing:** Start testing as early as possible.
- 4) **Defect Clustering:** Most defects are usually found in a small number of modules.
- 5) **The Pesticide Paradox:** Update tests regularly.
- 6) **Testing is Context Dependent:** Different software needs different testing methods (e.g., banking vs. gaming).
- 7) **Absence of Error Fallacy:** Even if no bugs are found, the product is useless if it doesn't meet requirements.

21) Difference between QA vs QC vs Tester

Quality Assurance	Quality Control	Testing
Activities which ensure the implementation of processes, procedures and standards in context of developed software and intended requirements.	Activities which ensure verification of developed software with respect to documented requirements.	Activities which ensure identification of bugs/errors/defects in the Software.
Focuses on processes and procedures rather than conducting actual testing on the system.	Focuses on actual testing by executing Software with intent to identify bug/defect through implementation of procedures and processes.	Focuses on actual testing.
Process oriented activities	Product oriented activities	Product oriented activities
Preventive activities	It is a corrective process	It is a preventive process

22) Difference between Smoke Testing and Sanity Testing?

Smoke Testing	Sanity Testing
Done for initial builds or stability when software is unstable.	Done for continuous updates or multiple rounds of updates (regression tests).
Checks critical functionalities — e.g., does the application start successfully?	Checks new functionality, bug fixes in the build.
Performed on Build 1, 2, 3...	Performed on Build 30, 31, 32...
Broad and shallow — covers major features.	Narrow and deep — covers specific area.

23) Difference between Verification and Validation

Verification	Validation
Refers to the set of activities that ensure software correctly implements the specific function.	Refers to the set of activities that ensure the software built is traceable to customer requirements.
Includes checking documents, designs, codes, and programs.	Includes testing and validating the actual product.
Done without executing code (Static Testing).	Done by executing the software (Dynamic Testing).

24) Explain Types of Performance Testing

- Load Testing
- Stress Testing

25) What is Error, Defect, Bug and Failure?

Term	Definition
Error	A mistake in coding is called an Error.
Defect	Error found by a tester is called a Defect.
Bug	Defect accepted by the development team is called a Bug.
Failure	Build does not meet the requirements — it is a Failure.

26) What is Bug Life Cycle?

The stages of the Bug Life Cycle are as follows:

- New → Assign → Open
- Open → (1) Not a Bug → Fixed → Pending Retest → Retest → Verified → Closed
- (2) Rejected
- (3) Deferred
- (4) Duplicated

27) Difference between Functional Testing and Non-Functional Testing

Functional Testing	Non-Functional Testing
Verifies the operations and actions of an application.	Verifies the behavior of an application.
Based on requirements of the customer.	Based on expectations of the customer.

Helps to enhance the behavior of the application.	Helps to improve the performance of the application.
Easy to execute manually.	Hard to execute manually.
Tests what the product does.	Describes how the product does it.
Based on business requirement.	Based on performance requirement.

28) To create HLR & TestCase of (Instagram , Facebook) first page and chat functionality .

HLR :

1	High Level Requirements (HLR) Table		
2	Functionality ID	Functionality Name	Description
3	Instagram Login		
4	HLR-01	Instagram Login	The system must allow users to log into their Instagram accounts using a valid phone number, username, or email along with the correct password.
5	Instagram Home Feed		
6	HLR-02	Instagram Home Feed	The system must display a personalized home feed showing posts and stories from the people the user follows.
7	Instagram Chat Functionality		
8	HLR-03	Instagram Chat Functionality	The system must allow users to send messages, receive messages, and manage their chat conversations (e.g., pinning a chat).
9	Facebook Login		
10	HLR-04	Facebook Login	The system must allow users to log into their Facebook accounts using their registered email or phone number and password.
11	Facebook Signup		
12	HLR-05	Facebook Signup	The system must allow new users to create an account by providing their first name, surname, email/mobile, password, date of birth, and gender.
13	Facebook Feed		
14	HLR-06	Facebook Feed	The system must allow logged-in users to view their news feed, interact with posts (like, comment), and discover new groups or channels.
15			

Test Cases:

	A	B	C	D	E	F	G	H	I	J
1	Detailed Test Case Table									
2	Functionality ID	Test Case Name	Test Summary	Pre-Condition	Test Steps	Test Data	Expected Result	Actual Result	Status	
3	Instagram Login									
4	INSTA-LOG-01	Successful Login	Verify user can log in with valid credentials	User must have a registered Instagram account	1. Open Instagram Login page 2. Enter username 3. Enter	Username: valid_user Password: ValidPass1 23!	User should be successfully logged in and redirected to Home Feed	As per expected result	Pass	
5	INSTA-LOG-02	Invalid Password	Verify login fails when password is wrong	User must have a registered account	1. Open Instagram Login page 2. Enter correct username 3. Enter incorrect password 4.	Username: valid_user Password: wrongpass!	System should show error: " Sorry, your password was incorrect."	As per expected result	Pass	
6	INSTA-LOG-03	Empty Credentials	Verify login fails if fields are empty	User is on the login page	1. Leave username and password blank 2. Click 'Log In'	Username: [Blank] Password: [Blank]	The 'Log In' button should be disabled or an error should prompt to fill fields.	As per expected result	Pass	
7	INSTA-LOG-04	Forgot Password	Verify 'Forgot password?' link works	User is on the login page	1. Click on 'Forgot password?' link	N/A	User is redirected to the password reset page.	As per expected result	Pass	
8	Instagram Home Feed									
9	INSTA-FEED-01	Load Feed	Verify home feed loads posts	User is logged into Instagram	1. Navigate to the Home screen 2. Wait for the feed to load	N/A	Posts and stories from followed accounts should appear.	As per expected result	Pass	
10	INSTA-FEED-02	Like a Post	Verify user can 'Like' a feed post	Feed is loaded with posts	1. Find a post on the feed 2. Click on the Heart icon below the post	N/A	The Heart icon should turn red, indicating the post is liked.	As per expected result	Pass	
11	Instagram Chat Functionality									
12	INSTA-CHAT-01	Send Message	Verify user can send a message	User is logged in and has internet	1. Click the Messenger icon 2. Open a chat 3. Type a message 4. Tap Send	Message text: "Hello!"	The message should appear in the chat as sent.	As per expected result	Pass	
13	INSTA-CHAT-02	Pin Chat	Verify user can pin a chat conversation	User is in the messages list	1. Swipe left / long-press a chat 2. Click the 'Pin' option	Any active chat	The chat should be moved and pinned to the top of the message list.	As per expected result	Pass	
< > Table 1Table 2 + ReadyAccessibility: Investigate										

	A	B	C	D	E	F	G	H	I
14	INSTA-CHAT-03	Blank Message	Verify user cannot send an empty message	User is inside a chat conversation	1. Open a chat 2. Do not type anything 3. Look for the Send button	Message: [Blank]	The 'Send' button should be hidden or disabled.	As per expected result	Pass
15	Facebook Login								
16	FB-LOG-01	Valid Login	Verify valid Facebook login	User has a valid Facebook account	1. Enter email/phone 2. Enter password 3. Click 'Log In'	Email: test@email.com Pass: Valid123	User is logged in and sees the Facebook Feed.	As per expected result	Pass
17	FB-LOG-02	Unregistered Email	Verify login with unregistered email	User is on Facebook login page	1. Enter an email not registered 2. Enter any password 3.	Email: fake@fake.com Pass: pass123	Error message should display indicating account does not exist.	As per expected result	Pass
18	Facebook Signup								
19	FB-SIGN-01	Valid Signup	Verify successful new account creation	User does not have an account	1. Go to signup 2. Fill all required fields correctly 3. Click 'Sign Up'	First: John Last: Doe Email: new@eq.com DOB: 10/10/2000	Account is successfully created and welcome page is shown.	As per expected result	Pass
20	FB-SIGN-02	Blank First Name	Verify signup validation for first name	User is on signup page	1. Leave 'First name' blank 2. Fill other fields 3. Click 'Sign Up'	First name: [Blank]	A red error warning should appear asking for the first name.	As per expected result	Pass
21	FB-SIGN-03	Invalid Email Format	Verify signup email format validation	User is on signup page	1. Enter all details 2. Enter an invalid email structure 3. Click 'Sign Up'	Email: johndoe123	Error should indicate that a valid email address is required.	As per expected result	Pass
22	Facebook Feed								
23	FB-FEED-01	Post Status	Verify user can create a post	User is on FB Feed	1. Click "Write something..." 2. Type a sentence 3. Click 'Post'	Text: "Enjoying my QA class!"	The new post should appear at the top of the user's news feed.	As per expected result	Pass
24	FB-FEED-02	Join Group/Stream	Verify user can join a channel/stream	User is scrolling FB Feed	1. Look for a suggested stream	"Sunday Stream" post	User should successfully join the stream/group.	As per expected result	Pass
25	FB-FEED-03	Offline Feed	Verify feed behavior offline	User is logged in, but internet is OFF	1. Turn off Wi-Fi/Data 2. Pull to refresh the feed	N/A	App should show a "No Internet Connection" banner or warning.	As per expected result	Pass
	FB-FEED-04	React to Post	Verify user can react to a feed post	User is scrolling FB Feed	1. Find a friend's post 2. Long press "Like" button 3. Select	N/A	The Heart reaction should register on the chosen	As per expected result	Pass

29) To create HLR & TestCase of WebBased (WhatsApp web) and Whatsapp Group Chat.

	A	B	C
1	High Level Requirements (HLR) Table		
2	Functionality_id	Functionality Name	Description
3	WhatsApp Web & Groups		
4	HLR-01	Authentication/Login	The system must allow users to log in securely by scanning a unique QR code from the WhatsApp application on their mobile phone.
5	HLR-02	Chat Functionality	The system must allow users to send, receive, and delete text messages, emojis, and media files (documents, images, videos) to individual contacts.
6	HLR-03	Group Chat	The system must allow users to participate in group chats, tag/mention other members, and allow group administrators to add or remove participants.
7	HLR-04	Logout/Session Management	The system must allow the user to log out securely from the web interface or remotely using their mobile phone to protect their privacy.
8			
9			
10			

	A	B	C	D	E	F	G	H	I	J
1	Detailed Test Case Table									
2	Functionality ID	Test Case Name	Test Summary	Pre-Condition	Test Steps	Test Data	Expected Result	Actual Result	Status	
3	WhatsApp Web & Groups									
4	WhatsApp – Authentication/Login									
5	WA-LOG-01	Valid QR Login	Check login with a valid QR code	Web page is open	1. Open web.whatsapp.com 2. Open WhatsApp on phone 3. Go to Linked Devices 4. Scan QR code	Valid QR code	User is logged in successfully.	As per expected result	Pass	
6	WA-LOG-02	Expired QR Login	Check login behavior when QR code expires	Web page is idle	1. Open web.whatsapp.com 2. Wait for QR expiration 3. Try to scan	Expired QR code	System rejects scan; prompts reload.	As per expected result	Pass	
7	WA-LOG-03	Keep Signed In	Check session persistence	Web page is open	1. Check 'Keep me signed in' 2. Scan QR 3. Reopen tab	-	User remains logged in.	As per expected result	Pass	
8	WA-LOG-04	Unauthorized Access	Check direct access without login	Logged out	1. Enter chat URL directly	web.whatsapp.co m/chat	Redirects to login page.	As per expected result	Pass	
9	WhatsApp – Chat Functionality									
10	WA-CHAT-01	Send Text Message	Check sending text message	Logged in	1. Select contact 2. Type message 3. Click Send	"Hello QA"	Message sent with timestamp.	As per expected result	Pass	
11	WA-CHAT-02	Send Empty Message	Check sending spaces	Logged in	1. Select contact 2. Type spaces 3. Send	" "	Send button disabled.	As per expected result	Pass	
12	WA-CHAT-03	Send Emoji	Check sending emojis	Logged in	1. Select contact 2. Select emoji 3. Send	Smiley	Emoji sent and displayed.	As per expected result	Pass	
13	WA-CHAT-04	Send Document	Check uploading document	Logged in	1. Click Attach 2. Choose Document	<5MB PDF	Document sent successfully.	As per expected result	Pass	
14	WA-CHAT-05	Large File Upload	Check large file limit	Logged in	1. Click Attach 2. Select >3GB video	>3GB Video	Error: "File selected is too large".	As per expected result	Pass	
15	WA-CHAT-06	Delete Message	Delete for everyone	Logged in	1. Send message 2. Select 'Delete for everyone'	Text	Message replaced with "You deleted this".	As per expected result	Pass	
16	WhatsApp – Group Chat									
17	WA-GRP-01	Create Group	Check group creation	Logged in	1. Tap '+' New group 2. Add contacts 3. Set name 4. Create	"QA Project"	Group created successfully.	As per expected result	Pass	

[illegible]

30) Create Test Cases on Compose Mail Functionality.

QA Testing Assignment: Gmail – Compose Email

Detailed Test Case Table | Module: Compose Window

Functional ity ID	Test Case Name	Test Summary	Pre-Condition	Test Steps	Test Data	Expected Result	Actual Result	Status
Compose Email – Functionality								
201	Open Compose Window	Verify compose window opens	User logged into Gmail	Click Compose	N/A	Compose window opens	Compose window opened at bottom-right corner with fields (To, Subject, Body) visible	Pass
202	Enter Recipient Email	Verify email input	Compose open	Enter email in "To"	test@gmail.com	Email accepted	Email accepted and displayed as recipient chip	Pass
203	Invalid Email Format	Verify invalid email validation	Compose open	Enter invalid email	test@com	Error message shown	Red validation border shown and email not accepted	Pass
204	Add Multiple Recipients	Verify multiple emails	Compose open	Enter multiple emails	a@gmail.com, b@gmail.com	All accepted	Multiple recipient chips created successfully	Pass
205	CC Field Functionality	Verify CC works	Compose open	Click CC → enter email	cc@gmail.com	Email added	CC field expanded and email added successfully	Pass
206	BCC Field Functionality	Verify BCC works	Compose open	Click BCC → enter email	bcc@gmail.com	Email added	BCC field expanded and email added successfully	Pass
207	Enter Subject	Verify subject input	Compose open	Enter subject	Test Mail	Subject added	Subject text displayed correctly in subject field	Pass
208	Empty Subject Warning	Verify warning	Compose open	Leave subject → click send	N/A	Warning popup	Popup displayed: "Send this message without a subject?"	Pass
209	Enter Email Body	Verify typing message	Compose open	Type message	Hello	Message visible	Text entered successfully in email body editor	Pass
210	Send Email Successfully	Verify send email	Compose open	Fill all → click send	Valid data	Email sent	Message sent and "Message sent" toast notification displayed	Pass
211	Send Without Recipient	Verify validation	Compose open	Leave To → click send	N/A	Error shown	Error popup: "Please specify at least one recipient"	Pass
212	Attach File	Verify attachment	Compose open	Upload file	sample.pdf	File attached	File uploaded and visible with attachment icon	Pass

211	Send Without Recipient	Verify validation	Compose open	Leave To → click send	N/A	Error shown	Error popup: "Please specify at least one recipient"	Pass
212	Attach File	Verify attachment	Compose open	Upload file	sample.pdf	File attached	File uploaded and visible with attachment icon	Pass
213	Attach Large File	Verify file size limit	Compose open	Upload large file	>25MB	Error shown	File uploaded to Google Drive and link shared automatically	Pass
214	Remove Attachment	Verify remove file	File attached	Click remove	N/A	File removed	Attachment removed successfully from compose window	Pass

Functional ity ID	Test Case Name	Test Summary	Pre-Condition	Test Steps	Test Data	Expected Result	Actual Result	Status
215	Formatting Text	Verify formatting	Compose open	Apply bold/italic	Text	Formatting applied	Text formatting applied correctly (bold/italic visible)	Pass
216	Insert Link	Verify link insertion	Compose open	Insert link	google.com	Link inserted	Hyperlink added and clickable in body	Pass
217	Save Draft Automatically	Verify auto-save	Compose open	Type → wait	N/A	Draft saved	Draft saved automatically and visible in Drafts folder	Pass
218	Close Compose Window	Verify save on close	Compose open	Click close	N/A	Draft saved	Compose closed and draft saved automatically	Pass
219	Discard Draft	Verify discard	Draft exists	Click discard	N/A	Draft deleted	Draft removed permanently after confirmation	Pass
220	Keyboard Shortcut Send	Verify Ctrl+Enter send	Compose open	Press Ctrl+Enter	Valid data	Email sent	Email sent successfully using keyboard shortcut	Pass

31) SDLC vs STLC — What's the Difference?

SDLC (Software Development Life Cycle)

SDLC is the complete process of planning, creating, testing, deploying, and maintaining a software product. It defines phases for the whole development team — not just testers.

- Phase 1: Requirements Gathering/Analysis
- Phase 2: System Design
- Phase 3: Implementation (Coding)
- Phase 4: Testing
- Phase 5: Deployment
- Phase 6: Maintenance

STLC (Software Testing Life Cycle)

STLC is a subset of SDLC. It defines specifically what the testing team does — from understanding requirements to closing all test activities. Every phase in STLC has entry criteria (what must be ready before you start) and exit criteria (what must be done before you finish).

- Phase 1: Requirement Analysis — Study requirements from a testing perspective.
- Phase 2: Test Planning — Plan what, how, who, and when to test. Create the Test Plan document.
- Phase 3: Test Case Development — Write detailed test cases and prepare test data.
- Phase 4: Test Environment Setup — Set up servers, browsers, devices, databases needed for testing.
- Phase 5: Test Execution — Run tests, log results, report bugs.
- Phase 6: Test Cycle Closure — Analyze what was tested, prepare closure report, and document lessons learned.

Key Difference in One Line

SDLC covers the entire product development. STLC covers only the testing activities within that development.

Think of SDLC as the full recipe book, and STLC as just the chapter on 'Quality Check' within that book.

32) What is a Test Plan?

A Test Plan is a document that describes the scope, approach, resources, and schedule of all testing activities for a project. It is like a roadmap for the testing team.

What Does a Test Plan Include?

- Test Objectives — What are we trying to achieve through testing?
- Scope of Testing — Which features will be tested and which won't.
- Test Strategy/Approach — How testing will be done (manual, automated, combination).
- Entry and Exit Criteria — When testing starts and when it stops.
- Resource Planning — Who will test? What tools and environments are needed?
- Risk Assessment — What can go wrong, and how to handle it.
- Test Schedule — Timeline for each testing phase.
- Test Deliverables — What documents will be produced (test cases, bug reports, closure report).
- Test Environment — Browsers, OS, devices, databases to be used.

A Test Plan is prepared by the Test Lead/Senior QA Manager and is continuously updated as the project evolves.

33) Test Scenario vs Test Case vs Test Script

Test Scenario

A Test Scenario is a high-level description of what needs to be tested. It answers the question 'WHAT to test?' It is like a topic you want to verify.

Example: Test Scenario: Verify that a user can log into the application using valid credentials.

Test Case

A Test Case is a detailed, step-by-step document that answers 'HOW to test?' It includes specific input values, the steps to follow, and the expected result.

Example: Test Case: Step 1 — Open login page. Step 2 — Enter username 'john@test.com'. Step 3 — Enter password 'Pass@123'. Step 4 — Click Login. Expected Result: User is redirected to dashboard.

Every test case has: Test Case ID, Module, Pre-Conditions, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail status.

Test Script

A Test Script is a set of sequential instructions that detail how to execute a business function. It is written in enough detail that someone else (not the script writer) can execute it without confusion. Test scripts can be manual (written in Excel) or automated (written in Selenium, etc.).

Quick Summary

- Test Scenario = WHAT to test (broad topic)
- Test Case = HOW to test (step-by-step details)
- Test Script = Complete executable sequence of instructions (manual or automated)

34) Priority vs Severity

What is Severity?

Severity is about the **IMPACT** of the bug on the system. It is customer-focused and technical. It defines how seriously the bug affects the functionality.

- Critical — The entire system crashes. No workaround exists. Example: Login button does nothing.
- Major (High) — Major functionality is broken, but a workaround exists.
- Moderate (Medium) — The system gives incorrect or incomplete results but doesn't crash.
- Minor (Low) — Doesn't affect functionality but causes minor inconvenience.
- Cosmetic — Visual issue only. Example: Font color is slightly off.

What is Priority?

Priority is about **HOW URGENTLY** the bug needs to be fixed. It is business-focused and relative. It is decided based on customer requirements and business impact.

- Critical — Fix immediately! The business cannot function without this.
- High — Fix in the next release or build.
- Medium — Fix in normal course of development.
- Low — Fix when time permits. It's an irritant but not blocking.

The Classic Confusion: Difference Between Priority and Severity

These two are often confused. Here are the 4 real-world scenarios:

- High Priority + High Severity: Login button is broken on the main page. Must fix now. Business is down.
- High Priority + Low Severity: Company name is misspelled on the homepage. It looks bad publicly, so fix it fast — but technically it doesn't crash anything.
- Low Priority + High Severity: A feature that only 2 users use per year crashes the system. Severity is high (crashes system) but priority is low (rarely used).
- Low Priority + Low Severity: A spelling mistake in a footer. Nobody notices, nobody cares.

Remember: Severity = technical impact. Priority = business urgency. They are independent of each other.

35) Bug Categories

Bugs can be grouped into categories based on what part of the system they affect:

- 1. Database / Data Quality Defects:** When data is stored incorrectly in the database — wrong values inserted, null values where real values should be, or records not being deleted properly.
- 2. Critical Functionality Defects:** These bugs break the core functionality of the application. Example: Clicking the payment button throws an exception and the app crashes.
- 3. Functionality Defects:** Bugs that affect how features behave. Example: The 'Save' button doesn't save data. A filter doesn't filter correctly. JavaScript errors popping up.
- 4. Security Defects:** These are the most severe. They expose the system to attacks. Example: System accepts invalid username/password. Users can access admin pages without permission.
- 5. User Interface (UI) Defects:** Issues with the look and feel. Less severe technically. Example: Alignment problems, spelling mistakes on screen, wrong error message shown.

36) Bug Life Cycle (Defect Life Cycle)

When a bug is found, it goes through a series of states before it is closed. This journey is called the Bug Life Cycle.

- New — Tester finds a bug and logs it for the first time.
- Assigned — Test lead approves and assigns the bug to a developer.
- Open — Developer starts analyzing the bug.
- Fixed — Developer fixes the bug and marks it as 'Fixed.'
- Pending Retest — The fix is ready but waiting for the tester to verify.
- Retest — Tester is actively re-testing the fixed bug.
- Verified — Bug is confirmed fixed. No more issue.
- Closed — Bug is completely done.
- Reopened — Tester finds the bug still exists after the fix. Developer must fix it again.
- Rejected — Developer disagrees that it's a bug. Example: It's working as designed.
- Deferred — Bug will be fixed in a future release, not this one.
- Duplicate — Same bug was already reported by someone else.
- Not a Bug — It doesn't affect functionality. Might be a feature, not a defect.

37) Bugzilla — Bug Tracking Tool

Bugzilla is a free, open-source bug tracking system. It is written in Perl and uses a MySQL database. Testers use it to report, track, and manage bugs throughout the project.

Advantages of Bugzilla

- Free and Open Source — No license cost. Anyone can use it.
- Advanced Search — Easily find bugs by status, severity, priority, module, etc.
- Email Notifications — Automatically alerts team members when a bug status changes.
- Bug Reporting by Email — You can file or update bugs by just sending an email.
- Time Tracking — Track how long it takes to fix each bug.
- Strong Security — Controls who can see and edit which bugs.
- Customization — You can customize fields, workflows, and notifications.
- Localization — Supports multiple languages.
- Integration — Can be linked to other tools like Quality Center and TestLink.

38) Agile Methodology

Agile is a way of managing software projects by breaking the work into small pieces and delivering them in short cycles. The team constantly collaborates with the client and improves the product at every stage.

Agile is based on the iterative-incremental model — build a little, test a little, show the client, get feedback, repeat.

Agile Values (The Manifesto)

- Individuals and interactions OVER processes and tools — Talk to your team instead of just following a tool.
- Working software OVER comprehensive documentation — A working app matters more than 100 pages of docs.
- Customer collaboration OVER contract negotiation — Work WITH the client, not just for a contract.
- Responding to change OVER following a plan — Be flexible. Requirements can change.

Agile Principles (Key Ones)

- Deliver working software frequently — every 2 weeks to 1 month.
- Welcome changing requirements, even late in development.
- Business people and developers work together daily.
- The best architecture emerges from self-organizing teams.
- At regular intervals, teams reflect on how to become more effective.

39) Scrum — The Most Popular Agile Framework

Scrum breaks work into short periods called Sprints (usually 2–4 weeks). At the end of each sprint, working software is delivered to the client.

Scrum Roles:

- Product Owner — Communicates with clients, understands requirements, creates the Product Backlog.
- Scrum Master — Manages daily scrum meetings, removes obstacles for the team.
- Scrum Team — Developers, testers, designers who work together in the sprint.

Scrum Artifacts:

- Product Backlog — Complete list of all features to be built.
- Sprint Backlog — The small set of tasks picked for the current sprint.
- Burndown Chart — A graph showing how much work remains in the sprint.

Scrum Ceremonies:

- Sprint Planning — Team decides what to do in the upcoming sprint.
- Daily Scrum (Stand-up) — 15-minute daily meeting: What did I do yesterday? What will I do today? Any blockers?
- Sprint Review — Demo the completed work to the client.
- Sprint Retrospective — Team discusses what went well and what to improve.

31) Kanban

Kanban is another Agile framework. It uses a board with columns representing stages of work: To Do, In Progress, Testing, Done. Tasks are moved across columns. It is great when you want to release work continuously without fixed sprints.

40) Authentication vs Authorization + Web Testing Issues

Authentication

Authentication is the process of verifying WHO you are. It checks your identity — are you really the person you claim to be?

Example: Entering your username and password to log in. The system checks: 'Is this person registered?'

Authorization

Authorization is the process of verifying WHAT you are allowed to do. After authentication, the system checks your permissions.

Example: After logging in, a normal user can view their profile, but only an admin can delete users. The system checks: 'Does this person have permission to do this?'

In short: Authentication = Login check. Authorization = Permission check. You can be authenticated but not authorized.

Common Web Testing Problems

- Broken Links — Clicking a link takes you to a 404 Page Not Found.
- Authentication Issues — System accepts invalid usernames/passwords. Accounts can be accessed without proper credentials.
- Authorization Issues — Regular users can access admin pages. Users can see other users' data.
- Incorrect Form Validations — Forms accept invalid email formats. Password fields accept spaces or very short passwords.
- Session Timeout Issues — Sessions expire too quickly or not at all. After logout, pressing Back still shows private pages.
- Cross-Browser Compatibility — Website looks broken in Firefox but fine in Chrome.
- Performance Issues — Pages load slowly under normal traffic.
- HTTPS Not Used — Sensitive data being sent over HTTP instead of HTTPS.
- SQL Injection — Input fields not validated, allowing database attacks.
- CAPTCHA Not Working — Bots can submit forms freely.
- Responsive Design Issues — Website broken on mobile or tablet screens.
- Cookie and Session Handling — Cookies not expiring properly. Session data leaking.

41) When to Use Usability Testing

Usability Testing checks how easy and user-friendly the software is. It focuses on whether real users can actually use the system comfortably without confusion.

When Should You Use Usability Testing?

- When you want to check if users can find and use features intuitively.
- When launching a new product or redesigning an existing one.
- When users are complaining that the system is 'confusing' or 'hard to use.'
- When error messages are unclear and users don't understand what went wrong.
- When users are doing things differently than the designers expected.
- When you want to compare two different UI designs (e.g., A/B testing).

What Problems Does Usability Testing Catch?

- 'Where do I click next?' — Navigation is confusing.
- 'What does this icon mean?' — Icons or jargon are unclear.

- 'Why am I getting this error?' — Error messages are unhelpful.
- 'The session timed out too quickly.' — Session duration is insufficient.

Goals of Usability Testing

- Effectiveness — Can users complete the tasks?
- Efficiency — How quickly can they do it?
- Accuracy — How many mistakes do they make?
- User Satisfaction — Do they feel good using it?

Research shows that just 5 users can uncover about 80% of usability problems. You don't need hundreds of testers.

42) Procedure for GUI Testing

GUI (Graphical User Interface) Testing is the process of testing all the visual elements of the application — buttons, menus, icons, forms, dialog boxes, windows, and layouts.

What to Check in GUI Testing?

- Size, position, width, and length of all elements — are they correct per design?
- Input fields accept the right types of inputs.
- All buttons work as expected (Save, Cancel, Delete).
- Error messages are displayed correctly and in the right place.
- Font type, size, color, and alignment match the specification.
- Images are clear, properly aligned, and not distorted.
- Colors are aesthetically pleasing and meet contrast standards.
- Tab and Shift+Tab navigation order works correctly.
- Scrollbar appears only when content overflows.
- All pages have proper titles.
- Confirmation dialogs appear before any delete or update action.
- Hourglass/loading indicator appears when the app is busy.
- GUI works correctly on different screen resolutions.

Three Approaches for GUI Testing

1. Manual Testing: A tester manually checks each screen against requirements. Time-consuming but great for catching visual/design issues.

2. Record and Replay (Automation): An automation tool records your actions and replays them later. Tools like QTP/UFT or Selenium can be used. You record once, replay many times.

3. Model-Based Testing: A model (diagram) of the system's expected behavior is created. Test cases are automatically generated from this model. Useful for identifying unexpected states in the UI.

43) Test Scenarios for Various Objects & Applications

Below are practical test scenarios covering positive cases, negative cases, and edge cases for each object/application. These are written from a tester's perspective.

1. WhatsApp Chat Messages

- Verify that a text message is sent successfully to a contact.
- Verify that a message with emojis is sent and displayed correctly.
- Verify that an image/video is sent and received properly.
- Verify that a voice message can be recorded and sent.
- Verify that a message delivered tick (✓✓) appears after successful delivery.
- Verify that the blue tick (read receipt) appears after the recipient reads the message.
- Verify that a message can be deleted for yourself or for everyone.
- Verify that 'Replying to a message' shows the quoted original message.
- Verify that forwarding a message to another contact works.
- [Negative] Verify that a message cannot be sent when there is no internet.
- [Negative] Verify that a file larger than the allowed limit (e.g., 100MB) cannot be sent.
- [Edge] Verify that sending a very long text message (10,000 characters) works correctly.
- [Edge] Verify behavior when the recipient's phone is switched off.

2. Pen

- Verify that the pen writes smoothly on paper.
- Verify that ink flow is consistent without skipping.
- Verify that the cap fits properly and doesn't fall off.
- Verify that the pen does not leak ink when capped.
- Verify that the pen writes continuously for its rated life (e.g., 2 km of writing).
- [Negative] Verify what happens if the pen is pressed too hard — does it break?
- [Negative] Verify that the pen does not write on a wet surface.
- [Edge] Verify that the pen works in extremely cold weather.
- [Edge] Verify what happens when the pen runs out of ink.

3. Pen Stand

- Verify that the pen stand holds pens securely without tipping over.

- Verify that the stand holds the maximum number of pens advertised.
- Verify that it doesn't crack or break under normal weight.
- Verify that the material is durable (no rust for metal stands).
- [Negative] Verify what happens when you put more pens than the capacity.
- [Edge] Verify the stand remains stable on an uneven surface.

4. Door

- Verify that the door opens and closes smoothly.
- Verify that the door locks and unlocks correctly with the key.
- Verify that the hinges are properly aligned and don't squeak.
- Verify that the door handles work from both sides.
- Verify that the door provides proper insulation (no light gap).
- [Negative] Verify that the door cannot be opened without the correct key.
- [Negative] Verify what happens if the door handle breaks mid-use.
- [Edge] Verify the door works normally after extreme humidity or rain.

5. ATM (Automated Teller Machine)

- Verify that a valid debit card is accepted and PIN is verified.
- Verify that cash withdrawal works for a valid amount.
- Verify that the balance enquiry shows the correct balance.
- Verify that a transaction receipt is printed.
- Verify that the ATM returns the card after the session.
- [Negative] Verify that an invalid PIN shows an error and blocks after 3 attempts.
- [Negative] Verify that withdrawal is declined when balance is insufficient.
- [Negative] Verify that a blocked card is rejected with a proper message.
- [Negative] Verify behavior when the ATM is out of cash.
- [Edge] Verify that a transaction is handled properly if power fails mid-transaction.
- [Edge] Verify behavior when the receipt paper runs out.

6. Microwave Oven

- Verify that food heats evenly at default settings.
- Verify that the timer counts down correctly and stops at zero.
- Verify that the door latch works and the microwave stops when opened.
- Verify that power levels (low, medium, high) function correctly.
- Verify that the preset cooking modes (popcorn, pizza, etc.) work.
- [Negative] Verify that the microwave does not start with the door open.
- [Negative] Verify behavior when a metal container is placed inside.

- [Edge] Verify what happens when run with an empty interior.
- [Edge] Verify that it handles a power surge without damage.

7. Coffee Vending Machine

- Verify that inserting the correct coin/amount allows beverage selection.
- Verify that the selected coffee is dispensed correctly.
- Verify that the cup is properly placed before dispensing begins.
- Verify that the machine dispenses change correctly.
- [Negative] Verify that an insufficient amount shows an error and doesn't dispense.
- [Negative] Verify behavior when ingredients (coffee/milk) run out.
- [Edge] Verify that the machine handles simultaneous button presses gracefully.
- [Edge] Verify that the machine doesn't dispense if the cup is missing.

8. Chair

- Verify that the chair supports the maximum weight capacity without breaking.
- Verify that chair legs are stable on flat and uneven surfaces.
- Verify that armrests (if any) are properly attached.
- For office chairs: Verify that height adjustment works smoothly.
- For swivel chairs: Verify that the rotation is smooth and controlled.
- [Negative] Verify what happens when weight exceeds capacity.
- [Edge] Verify the chair doesn't crack or warp in extreme heat or cold.

9. Wrist Watch

- Verify that the watch displays the correct time after setting.
- Verify that the date function changes correctly at midnight.
- Verify that buttons for setting time work without sticking.
- Verify that the watch strap buckle holds securely.
- For digital: Verify alarm function works at the set time.
- [Negative] Verify behavior when the battery is low (should show battery indicator).
- [Negative] Verify that the watch is water-resistant up to its rated depth.
- [Edge] Verify correct time display during daylight saving transitions.

10. Lift / Elevator

- Verify that pressing a floor button takes the elevator to that floor.
- Verify that the door opens and closes automatically.
- Verify that the door reopens when the sensor detects an obstacle.
- Verify that the floor indicator displays the current floor.

- Verify that the emergency button triggers an alarm.
- Verify that the overload sensor prevents the door from closing when weight exceeds the limit.
- [Negative] Verify behavior when power is cut mid-ride — emergency lighting and intercom should work.
- [Edge] Verify that pressing multiple floors in sequence takes you to each floor.
- [Edge] Verify behavior when two people try to press contradictory floor buttons.

11. WhatsApp Payment

- Verify that a user can link a bank account successfully.
- Verify that sending money to a valid contact works correctly.
- Verify that the recipient receives the correct amount.
- Verify that a transaction notification is sent to both sender and receiver.
- Verify that the transaction history shows all past payments.
- [Negative] Verify that payment fails with insufficient bank balance.
- [Negative] Verify that an invalid UPI PIN shows an error message.
- [Negative] Verify that payment cannot be made to an unregistered number.
- [Edge] Verify behavior when the internet disconnects mid-transaction.
- [Edge] Verify the maximum transaction limit and ensure it's enforced.

12. Online Shopping — Flipkart

- Verify that a user can register and log in successfully.
- Verify that searching for a product returns relevant results.
- Verify that adding a product to cart works correctly.
- Verify that applying a valid coupon code gives the correct discount.
- Verify that placing an order with COD completes successfully.
- Verify that placing an order with a debit card completes with OTP verification.
- Verify that the order confirmation email/SMS is sent.
- Verify that the order tracking shows the current status.
- Verify that a return/refund request can be raised successfully.
- [Negative] Verify login fails with wrong password and shows error.
- [Negative] Verify checkout cannot proceed with an empty cart.
- [Negative] Verify that an expired coupon shows an appropriate error message.
- [Negative] Verify that payment fails gracefully when the bank server is down.
- [Edge] Verify behavior when the same product is added 100 times.
- [Edge] Verify search results when the query is a special character or blank.
- [Edge] Verify simultaneous order placement by two users for the last available item in stock.